

به نام خدا

پروژه سیستم مدیریت رستوران

نام درس : برنامه سازی پیشرفته

نام دانشجو : فرزاد وجودزاده

نام استاد : استاد ثبوتی

ترم : بهار 1405



معماری کلی برنامه

برنامه از 4 بخش اصلی تشکیل شده:

بخش اول << لایه دیتابیس :

- ❖ است SQLite مسئول اتصال به فایل Database کلاس
- ❖ دارد open(), close(), executeQuery(), createtable() متد های
- را از بقیه برنامه محفوظ میکند SQLite جزئیات

بخش دوم << لایه مدل :

- ❖ دارند person, restaurant, item, ... کلاس های
- ❖ فقط داده ها را به طور موقت نگه میدارند
- ❖ هستند getter & setter دارای

DAO بخش دوم << لایه

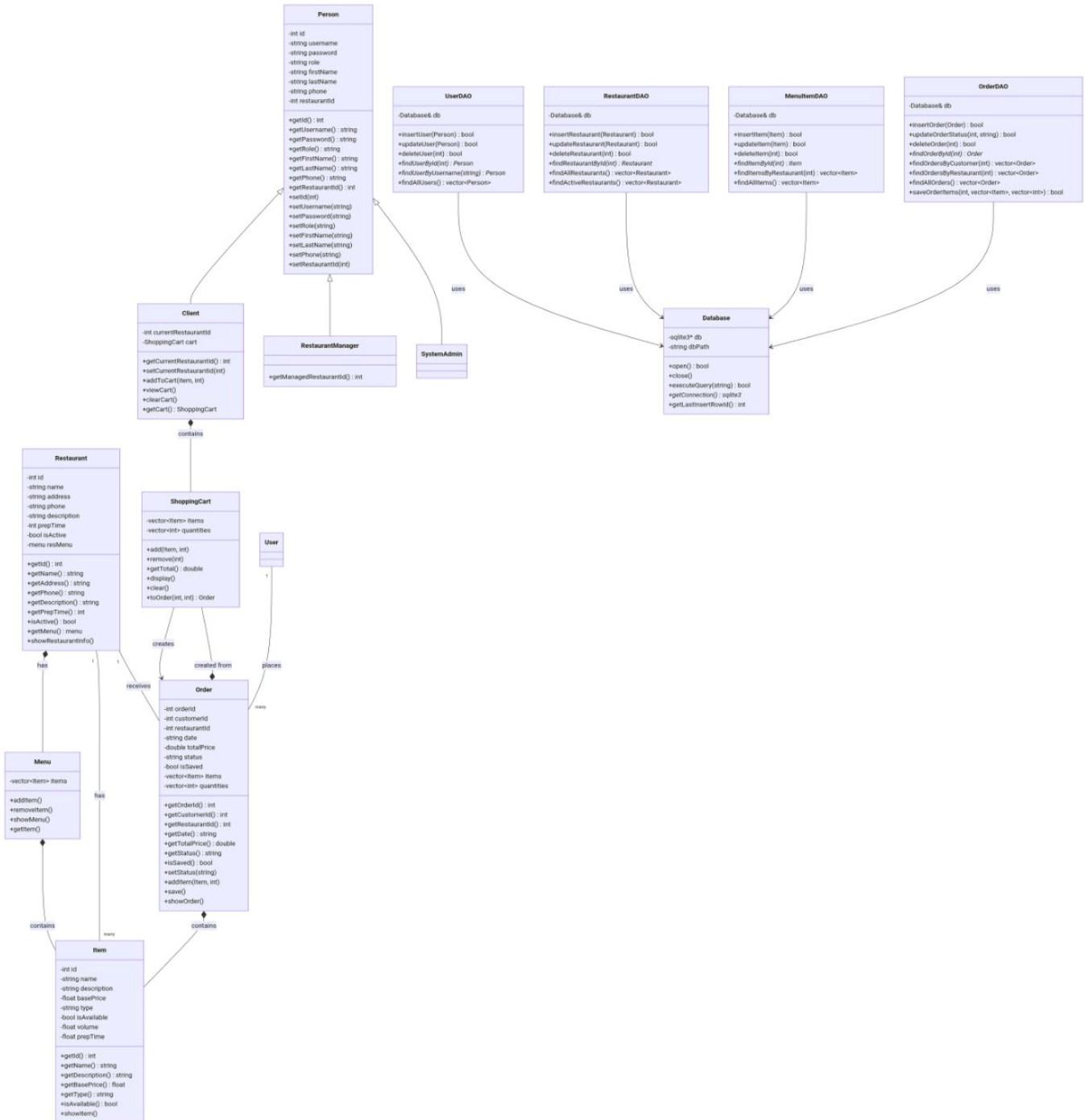
- ❖ restaurantDAO, menuitemDAO, orderDAO, userDAO 4 کلاس دارد که عبارتند از :
- ❖ مسئول ارتباط با دیتابیس هستند
- ❖ دارد Database(composition) یک ارجاع به شی از کلاس DAO هر
- ❖ دارند insert, update, delete, findbyid, findall , ... متد های

بخش سوم << لایه منو کنسولی :

- ❖ showmenuclient(), showmenuadmin(), showmenumanager(), signupmenu(), signinmenu(), firstmenu() را دارد
- ❖ مسئول نمایش منو ها و هدایت کاربر برای انجام کار
- ❖ ها برای گرفتن و ذخیره اطلاعات استفاده میکند DAO از

دیاگرام کلاس :

دیاگرام کلاس در شکل زیر نشان داده شده است توضیحات در ادامه داده خواهد شد



ارث بری :

ارث بری میکنند person از کلاس client , restaurantmanger , systemadmin کلاس های

ترکیب :

شامل یک ترکیب از userDAO , restaurantDAO, menuitemDAO, orderDAO کلاس های
هستند Database کلاس

هستند item شامل ترکیب کلاس menu, order, restaurant کلاس های

هست shoppingcart شامل ترکیبی از کلاس client کلاس

هست order شامل ترکیب کلاس shoppingcart کلاس

اصول شی گزایی به کار گرفته شده

کپسوله سازی :

کنترل میشن getter, setter هستند و با private تمام داده های همه کلاس ها

```
class person{
    private:
        string firstname;
        string lastname;
        string username;
        int id;
        string password;
        string phonnumber;
        string role;
        int resid;
```

ارث بری :

person از client, restaurantmanager, systemadmin کلاس های

ارث بری میکنند

```
class restaurantmanager : public person
{
public:
    restaurantmanager() : person() {}
    restaurantmanager(string nrol , string nfname, string nlname, string phone,
string nusername, string npass, int resid)
    : person(nrol , nfname, nlname, phone, nusername, npass){}
    ~restaurantmanager(){}
};
```

ترکیب :

در بخش قبل کاملاً توضیح داده شد یه مثال از ترکیب ببینیم :

```
class menu{
private:
    vector<item> items;
public:
    menu(){

    }
    void additem(string ntypeitem="", string nnameitem="", string
ndescriptionitem="", float nbacepriceitem=0, float detaile=0, bool
nisactive=true);
    void removeitemname(string nnameitem);
    void removeitemid(int nid);
    void showitemofmenu(int nid);
    void showmenu();
    item &getitemofmenu(int nid);
    ~menu(){

    }

};
```

پایگاه داده

بهره برده است که شامل 5 جدول است SQLite این برنامه از پایگاه داده

نام هر جدول و کویری مربوط به آن را خواهید دید

Users

```
CREATE TABLE IF NOT EXISTS users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT UNIQUE NOT NULL,  
    password TEXT NOT NULL,  
    role TEXT CHECK(role IN ('client', 'restaurant_manager', 'system_admin'))  
NOT NULL,  
    first_name TEXT NOT NULL,  
    last_name TEXT NOT NULL,  
    phone TEXT NOT NULL,  
    restaurant_id INTEGER DEFAULT -1  
);
```

Restaurants

```
CREATE TABLE IF NOT EXISTS restaurants(  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    address TEXT NOT NULL,  
    is_active INTEGER DEFAULT 1,  
    prep_time_minutes INTEGER NOT NULL,  
    phone TEXT NOT NULL,  
    description TEXT NOT NULL  
);
```

Menu_items



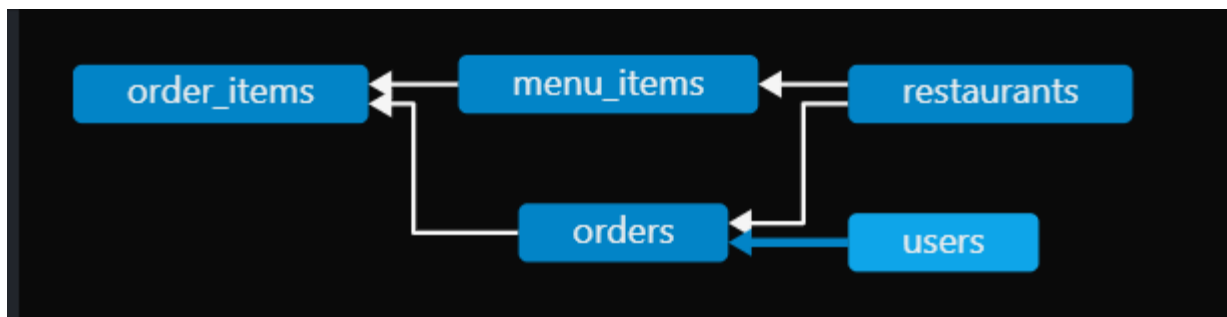
```
CREATE TABLE IF NOT EXISTS menu_items(  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    restaurant_id INTEGER NOT NULL,  
    name TEXT NOT NULL,  
    description TEXT NOT NULL,  
    base_price REAL NOT NULL,  
    type TEXT CHECK(type IN('drink', 'food', 'other')) NOT NULL,  
    is_active INTEGER DEFAULT 1,  
    volume REAL DEFAULT 0,  
    PREPTIME REAL DEFAULT 0,  
    FOREIGN KEY(restaurant_id) REFERENCES restaurants(id) ON DELETE CASCADE  
);
```

Orders

```
CREATE TABLE IF NOT EXISTS orders(  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    client_id INTEGER NOT NULL,  
    restaurant_id INTEGER NOT NULL,  
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    total_price REAL NOT NULL,  
    status TEXT CHECK(status IN('pending', 'preparing', 'ready_for_delivery',  
'delivered')) DEFAULT 'pending',  
    FOREIGN KEY(client_id) REFERENCES users(id),  
    FOREIGN KEY(restaurant_id) REFERENCES restaurants(id)  
);
```

Order_items

```
CREATE TABLE IF NOT EXISTS order_items(  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    order_id INTEGER NOT NULL,  
    menu_item_id INTEGER NOT NULL,  
    quantity INTEGER NOT NULL,  
    total_price REAL NOT NULL,  
    FOREIGN KEY(menu_item_id) REFERENCES menu_items(id),  
    FOREIGN KEY(order_id) REFERENCES orders(id) ON DELETE CASCADE  
);
```

vscode شکل مربوط به پایگاه داده

در شکل بالا به طور کامل رابطه بین این 5 جدول دیده میشو

و ارتباط با دیتابیس DAO لایه

این لایه پل ارتباطی برنامه با دیتابیس است و عملیات مربوطه رو انجام میده

در برنامه DAO ساختار لایه

عملیات	جدول مربوطه	DAO کلاس
Insert, delete, update, findbyid, findbyid, findbyroll, findall	users	UserDAO
Insert, delete, update, findbyresid, findall, findbystatus	restaurants	restarantDAO
Insert, delete, update, findbyid, findall, findbyresid	Menu_items	menuitemDAO
Insert, delete, update, findall, findbyid, findbyresid, findbyclientid, findbystatus	Orders, order_items	orderDAO

Database هر کلاس یه رفرنس به

callback این کلاس ها از توابع SELECT همینطور برای کویری های استفاده میکنند

را خواهید DAO در صفحه بعد یکی از این کد های یکی از کلاس های دید

```

class userDAO{
public:
    userDAO(database& ndb) : db(ndb){}

    bool insertuser(const person &user);
    bool updateuser(const person &user);
    bool removebyid(int d);
    person* findbyid(int id);
    person* findbyusername(string username);
    vector<person> findall();
    vector<person> findbyroll(string role);

private:
    static int callbackfindbyid(void* data, int argc, char** argv, char**
azcolname);
    static int callbackfindbyusername(void* data, int argc, char** argv,
char** azcolname);
    static int callbackfindall(void* data, int argc, char** argv, char**
azcolname);
    static int callbackfindbyrole(void* data, int argc, char** argv, char**
azcolname);
    database &db;
};

```

DAO مزایا استفاده از

- ❖ جداسازی منطق کد های دستابیس از برنامه
- ❖ تغییر در دیتابیس فقط در یک مکان نگهداری میشوند
- ❖ امنیت استفاده از پایگاه داده را زیاد میکند

مدیریت حافظه

new ها استفاده شده است آنهایی که با pointer در این برنامه از delete ساخته شده اند حتما

```
person* userDAO::findbyid(int id){
    person* user = new person;

    string query = "SELECT * FROM users WHERE id = " + std::to_string(id) + ";";

    char* errmsg = nullptr;
    int result;

    result = sqlite3_exec(db.getconnection(), query.c_str(), callbackfindbyid,
user, &errmsg);

    if(result != SQLITE_OK){
        std::cerr << "error in find user by id : " << errmsg << std::endl;
        sqlite3_free(errmsg);
        delete user;
    }

    return user;
}
```

مدیریت خطاها

در جاهایی از برنامه که انتظار میرفت کاربر داده غلط وارد کند به ویژه در بحث استفاده شده است `while` جداول دیتابیس از حلقه های

نبودن اشاره گر ها هم چک شده است تا برنامه خراب نشود `nullptr` حتی

```
while (true)
{
    if (pr->getpassword() == pass)
    {
        return pr;
        delete pr;
    }
    else
    {
        cout << "the password is not match!! enter correct pass\n";
        cin >> pass;
    }
}
```

```
while (true)
{
    if (ud.findbyusername(username) == nullptr)
    {
        cout << "your user name isn't found!! pleas try again or enter the word 'exit' to close programe\n";
        cin >> username;
        if (username == "exit")
        {
            break;
            return nullptr;
        }
    }
    else
    {
        break;
    }
}
```

سناریو ها

نتیجه مورد انتظار	سناریو	شماره
your restaurant is saved successfully!!	ثبت رستوران جدید	1
your user name isn't found!! pleas try again or enter the word 'exit' to close programe	ورود با نام کاربری نامعتبر	2
the item is added to your menu successfully!!	اضافه کردن ایتم به سبد خرید	3
your order is registered successfully!!	نهایی کردن سفارش	4

عکس های ار محیط برنامه

```
C:\Users\MAT\Desktop\MINIPROJECT>god.exe  
are you signed up? if yes enter 1 and no enter 2 (for exit from program enter 0):
```

منوی ثبت نام / ورود

```
*****the client menu*****  
enter the number of option you want(for exit inter 0)  
1.show all active restaurants  
2.choose restaurant  
3.show current restaurant menu  
4.add item to shopping cart  
5.show shopping cart  
6.check out order  
7.show history of orders
```

منوی مشتری

```
-----the shopping cart-----  
kebab x13 the price for per item: 120000  
the total price is : 1560000  
  
Press any key to continue . . .
```

سبد خرید

```
-----the manager menu-----  
enter the number your choosen option:  
1.view my restaurant informations  
2.edit restaurant information  
3.manag the menu  
4.view orders  
5.chang the status of order(you must know your order id)  
0.for exit program
```

منوی مدیر رستوران

```
choose an option (enter the number of option)
1.show all restaurants
2.active/disactive restaurant(you must have id of restaurant)
3.view salse report
4.view all users (enter 0 for exit)
```

منوی ادمین

```
===== the restaurants =====
id = 1 ..... name = shahrzad         active
id = 2 ..... name = mah shab         active
id = 3 ..... name = alone            not active
id = 4 ..... name = farzadkade       active
Press any key to continue . . .
```

لیست رستوران ها

```
===== salse report =====
total salse (all orders): 5010000 tomans
-----
orders by status:
-pending: 6 orders
-preparing: 1 orders
-ready for delivery: 1 orders
-delivered: 1 orders
-----
total orders: 9
total users: 12
total restaurants: 4
=====
Press any key to continue . . .
```

```
+++++++the history orders+++++++
=====factor=====
order id : 1
date : 2026-06-06 17:24:38
status : preparing
total base price is : 600000
total price you must pay is : 660000
=====
=====factor=====
order id : 2
date : 2026-06-06 17:47:08
status : ready_for_delivery
total base price is : 500000
total price you must pay is : 550000
=====
```

گزارش فروش

تاریخچه سفارش ها

نتیجه گیری

سیستم مدیریت آنلاین سفارش غذا با موفقیت پیاده سازی شده است. اصول شی گزایی (کپسوله سازی / ترکیب / ارپ بری) به کار گرفته شده است برای ذخیره اطلاعات بهره برده شده است SQLite از پایگاه داده برای

قابلیت های پیاده سازی شده

ثبت نام و ورود کاربران با سه نقش (مشتری / مدیر رستوران / ادمین)

مدیریت رستوران ها و منوی غذا

سبد خرید و ثبت سفارش

مشاهده تاریخچه سفارش ها

گزارش فروش کلی برای ادمین

مدیریت منو و سفارش ها برای مدیر رستوران

لینک گیت هاب

این پروژه در گیت هاب وجود دارد و در عملیات پیاده سازی آن از گیت هاب بهره برده شده است لینک گیت هاب پروژه در پایین قرار دارد

[farzadvojoud963-stack/MINIPROJECT: a system restaurant management with C++ language and sqlite.](https://github.com/farzadvojoud963-stack/MINIPROJECT)

توجه شما

